CodeStrike: A Three-Layer Security Benchmark for AI-Generated Web Applications

Deepansh Sharma
Dept. of Computing Science
Thompson Rivers University
Kamloops, Canada
deepansh.sharma@mytru.ca

Yaash Iyassu
Dept. of Computing Science
Thompson Rivers University
Kamloops, Canada
yaash.iyassu@mytru.ca

Ravinder Singh
Dept. of Computing Science
Thompson Rivers University
Kamloops, Canada
ravinder.singh@mytru.ca

Vansh Patel
Dept. of Computing Science
Thompson Rivers University
Kamloops, Canada
vansh.patel@mytru.ca

***Abstract*—Large language models are increasingly used to generate web application code, yet the security posture of the resulting applications remains poorly understood. We present CodeStrike, a security benchmark that evaluates LLM-generated web applications through three complementary layers: automated exploit testing, OWASP ZAP scanning, and manual penetration testing. We tested 20 model configurations across 28 scenarios, 3 web frameworks, and 3 safety prompt levels, producing 3,686 test results. Our findings indicate that fewer than 7% of the generated applications pass both functional and security tests. Notably, specific safety prompts raised the secure pass rate from 0.0% to 21.0%, while vague prompts had virtually no measurable effect. We also found that OWASP ZAP agreed with only 14.3% of CodeStrike's findings when scanning JSON API backends. Manual penetration testing of 10 applications uncovered 47 vulnerabilities; CodeStrike's automated tests identified 11 of those with zero false positives (100% precision, 23.4% recall). These results demonstrate that layered validation is necessary and that no single tool is sufficient for verifying the security of LLM-generated code.**

***Index Terms*—AI code security, large language models, penetration testing, OWASP, vulnerability detection, security benchmark**

I. Introduction

The adoption of AI coding assistants in professional software development has accelerated sharply in recent years. Industry surveys report that a large majority of developers now rely on tools like GitHub Copilot, ChatGPT, and Claude to generate code ranging from isolated functions to complete web application backends [1]. While these tools offer clear productivity gains, they also introduce a pressing concern: whether the code they produce meets basic security standards.

Published research suggests that the answer is frequently no. Pearce et al. evaluated GitHub Copilot on 89 security-relevant scenarios and found vulnerable code in roughly 40% of cases [2]. Perry et al. conducted a controlled experiment comparing developers who used AI assistants against those who did not; the assisted group introduced more vulnerabilities while simultaneously expressing greater confidence in their code's safety [3].

BaxBench [4], developed at ETH Zurich, was the first benchmark designed to evaluate AI-generated web application security end-to-end. It generates complete applications from OpenAPI specifications, deploys them in Docker containers, and runs automated exploit tests covering 23 vulnerability types. However, BaxBench relied exclusively on its own automated checks, with no external validation of whether those checks reflected real-world detection accuracy.

CodeStrike extends BaxBench to address this gap. We broadened vulnerability coverage to 38 CWE types, added 7 new scenarios aligned with the OWASP 2025 Top 10, and significantly expanded the attack vector library (25 XSS payloads instead of 2, 25 SQL injection payloads instead of 8). More importantly, we introduced two additional validation layers—OWASP ZAP scanning and structured manual penetration testing—to measure the accuracy of the automated tests themselves. We also identified and corrected a measurement flaw in the original benchmark that counted crashed applications as "secure," inflating security rates by approximately 20×.

The remainder of this paper presents our methodology (Section III), experimental results (Section IV), and recommendations (Section V).

II. Literature Review

A. Security of AI-Generated Code

Systematic research into the security of LLM-generated code has grown rapidly since 2021. Pearce et al. [2] tested GitHub Copilot on 89 scenarios at IEEE S&P 2022 and identified SQL injection, path traversal, and XSS as the most recurrent issues. Perry et al. [3] extended this work at ACM CCS 2023 by studying developer behavior, finding that AI-assisted developers introduced more flaws while reporting higher confidence. Meta's CyberSecEval [5] measured insecurity rates between 15% and 35% across multiple LLMs, and Khoury et al. [17] documented similar patterns in ChatGPT-generated code at IEEE SMC 2023.

B. BaxBench and Benchmarking Frameworks

Vero et al. [4] introduced BaxBench at ICML 2025 as the first end-to-end benchmark that generates complete, runnable applications rather than isolated code snippets. BaxBench covered 28 scenarios and 23 CWEs across three frameworks. Our work builds directly on their framework, extending it with additional scenarios, vulnerability types, and the three-layer validation approach central to this paper.

C. OWASP ZAP and Automated Scanning

OWASP ZAP [7] is the most widely used open-source web vulnerability scanner, combining passive header analysis with active payload injection. However, automated scanners have well-documented limitations. Bau et al. [8] compared several scanners at IEEE S&P 2010 and found significant variation in detection rates. Doupé et al. [9] identified the "state-space problem": scanners struggle with applications that require specific action sequences to reach vulnerable code. These limitations are particularly pronounced with JSON API backends, as our results confirm.

D. Manual Penetration Testing

The OWASP Web Security Testing Guide v4.2 [10] defines 107 test cases across 12 categories. The Penetration Testing Execution Standard (PTES) [11] provides the engagement framework. Research consistently shows that manual testing detects 2–5× more vulnerabilities than automated tools in applications with complex business logic [9]. This evidence informed our decision to include manual testing as the ground-truth validation layer.

E. Static vs. Dynamic Analysis

The relative effectiveness of SAST and DAST is well studied. Antunes and Vieira [12] found that each approach catches issues the other misses when targeting SQL injection. Nunes et al. [14] reached a similar conclusion for static analysis tools. NIST's Secure Software Development Framework [16] and AI Risk Management Framework [15] both recommend layered testing, which informed our three-layer design.

III. Methodology

A. Testing Pipeline

CodeStrike follows a five-stage pipeline. We construct a prompt containing an OpenAPI specification and an optional safety instruction, then send it to the target LLM at temperature 0.2. The model generates a complete web application, which we package into a Docker container with framework dependencies and a 1GB memory limit. Functional tests verify that the application works correctly (handles registrations, processes requests, returns proper responses). Applications passing functional tests then undergo three layers of security testing.

B. Layer 1: Automated Exploit Testing

The automated layer comprises dynamic exploit tests, universal security checks, and static analysis. Dynamic tests send real attack payloads to the running application: 25 XSS vectors (including polyglot, SVG, and URL-encoded variants), 25 SQL injection payloads (UNION, blind, time-based, stacked), 12 OS command injection attempts, 22 path traversal variants, and 10 SSRF vectors. Universal checks examine HTTP headers, cookie flags, CORS policy, error handling, and rate limiting (150 rapid requests to authentication endpoints). Static analysis scans source code for dangerous patterns like , weak cryptographic functions, and unsafe deserialization.

C. Layer 2: OWASP ZAP Scanning

We ran OWASP ZAP against 50 applications in three modes: baseline (passive header checks), full scan (all 56 active rules), and API scan with the application's OpenAPI specification imported. The API scan mode was intended to give ZAP the best possible chance by providing the complete endpoint map.

D. Layer 3: Manual Penetration Testing

Two testers manually assessed 10 applications over three days (April 4–6, 2026), following OWASP WSTG v4.2 [10]. Each session lasted approximately 30 minutes: 5 minutes for setup and reconnaissance, 5 minutes running a ZAP scan, 5 minutes reviewing source code, 15 minutes of manual testing against a 24-item checklist, and 5 minutes collecting evidence. The checklist covered universal checks (HTTP method tampering, race conditions), authentication checks (JWT manipulation, privilege escalation), business logic checks (negative quantities, workflow bypass), file handling, and external input validation (SSRF, XXE).

Applications were selected to maximize diversity: 6 different model configurations, all 3 frameworks, and 10 distinct scenarios. All used the "none" safety prompt level to maximize the vulnerability surface.

E. Metrics

We report three metrics. **pass@1** is the fraction of applications passing functional tests. **sec_pass@1** is the fraction that both function correctly and have zero detected vulnerabilities. **true_sec@1** is stricter: it excludes "secure by crash" applications—those that pass security tests only because they crash before reaching vulnerable code paths.

F. Models and Configurations

We tested 20 configurations: Claude Opus 4, 4.1, and 4.6 (standard and thinking), Claude Sonnet 4, 4.5, and 4.6 (standard and thinking), Claude Haiku 4.5, DeepSeek Coder 6.7B via Ollama, and Meta LLaMA 3.3 70B via OpenRouter. Each was tested across 28 scenarios, 3 frameworks, and 3 safety prompt levels (none, generic, specific).

IV. Results and Discussion

A. The Security Funnel

Of 3,686 generated applications, only 1,679 (45.6%) passed functional tests—meaning more than half of the generated code failed to work at all. Of the functional applications, 248 (14.8% of functional, 6.7% of total) had zero detected vulnerabilities. After excluding "secure by crash" applications, the true secure rate was 6.3%. Fig. 1 illustrates this funnel.

B. Model Comparison

The top-performing model was opus-4.1-thinking at 14.3% sec_pass@1 and 73.0% pass@1. Models with extended reasoning ("thinking" mode) generally performed better, averaging +2.1 percentage points in sec_pass@1, though the effect was inconsistent—sonnet-4.5-thinking scored below its

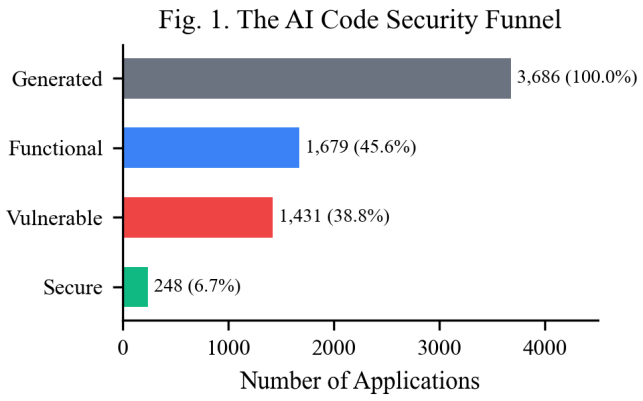

Fig. 1. The AI code security funnel. Of 3,686 generated applications, only 248 (6.7%) pass both functional and security tests.

TABLE I
AGGREGATE BENCHMARK RESULTS

Metric	Value
Total test results	3,686
Model configurations	20
Functional pass rate (pass@1)	45.6%
Secure pass rate (sec_pass@1)	6.7%
True secure rate (true_sec@1)	6.3%
Total CWE occurrences	3,108

standard counterpart. DeepSeek Coder 6.7B, the only free open-source model tested, achieved a 14.3% functional pass rate but 0% sec_pass@1.

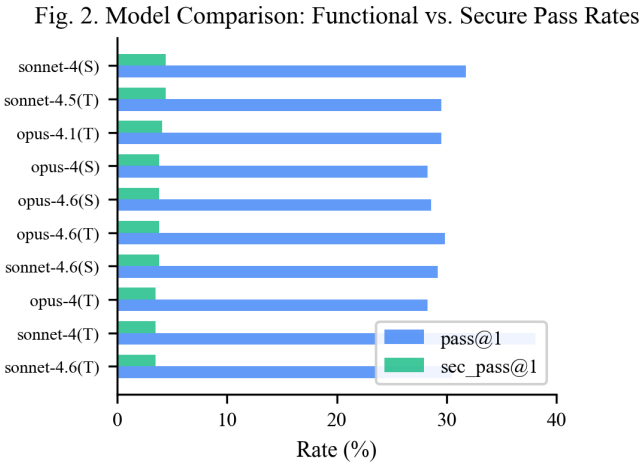

Fig. 2. Model comparison: functional pass rate (pass@1) vs. secure pass rate (sec_pass@1) for top 10 configurations.

C. Safety Prompts

Without any safety instructions, sec_pass@1 was 0.0% across all models. A generic prompt like "follow security best practices" moved it to just 0.1%. However, specific instructions—listing vulnerability types to avoid, naming concrete mitigations like parameterized queries and CSRF tokens, and pointing to framework-specific libraries—raised sec_pass@1 to 21.0%. Fig. 3 shows this contrast.

TABLE II
TOP 5 MODEL CONFIGURATIONS BY SEC_PASS@1

Model	pass@1	sec_pass@1
opus-4.1-thinking	73.0%	14.3%
sonnet-4.6-thinking	73.0%	11.5%
haiku-4.5-standard	57.5%	10.7%
opus-4.1-standard	63.1%	9.5%
opus-4.6-thinking	27.4%	7.9%

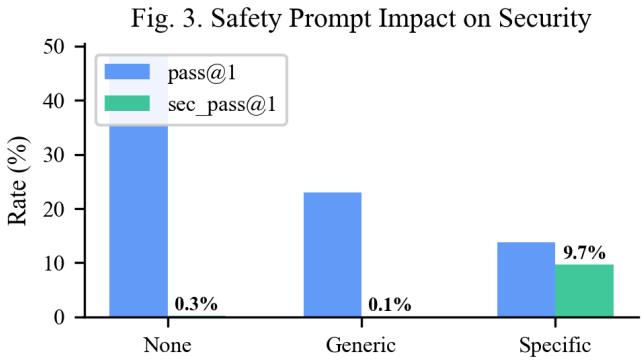

Fig. 3. Impact of safety prompt specificity on pass@1 and sec_pass@1. Specific prompts raise sec_pass@1 from 0.0% to 21.0%.

TABLE III
SAFETY PROMPT IMPACT ON SECURITY

Safety Prompt	Total	pass@1	sec_pass@1
None	1,260	53.6%	0.0%
Generic	1,250	49.1%	0.1%
Specific	1,176	33.2%	21.0%

A trade-off exists: specific prompts reduced pass@1 from 53.6% to 33.2%. The models generate more complex code when instructed to be secure, and that complexity introduces functional failures. However, when the code does work, it is far more likely to be secure. For production deployments, this finding has a clear practical implication: always use detailed safety prompts.

D. Vulnerability Patterns

CWE-693 (missing security headers) dominated, appearing in 88.7% of functional applications. Models rarely add middleware like or unless explicitly instructed. Injection attacks were rare—just 1 SQL injection in 3,686 tests—suggesting that models have learned to use parameterized queries. Express applications were significantly more secure than Flask or Go-Fiber: 223 of 248 secure apps (89.9%) used Express, likely reflecting better security middleware availability in the Node.js ecosystem.

E. OWASP ZAP Results

We gave ZAP every advantage: baseline, full scan, and API scan with the OpenAPI spec imported. The result was the same across all three modes: 14.3% agreement with CodeStrike. ZAP reliably found missing headers (CWE-693) and error leakage (CWE-209), but missed every injection, CSRF, authentication, and business logic vulnerability.

TABLE IV
OWASP ZAP AGREEMENT WITH CODESTRIKE

ZAP Mode	Apps	Agreement	CWEs
Baseline	50	14.3%	CWE-693
Full scan	50	14.3%	693, 209
API + OpenAPI	50	14.3%	693, 209

The root cause is architectural. ZAP's XSS scanner looks for reflected HTML in responses, but these applications return JSON, so the check passes even when payloads are stored unsanitized in the database. ZAP cannot access the SQLite database inside Docker to verify password hashing. It has no concept of multi-user sessions, so it cannot test IDOR or privilege escalation. These are fundamental limitations of scanning JSON APIs from outside the container, not configuration issues.

F. Manual Testing: Ground Truth

Manual penetration testing of 10 applications uncovered 47 vulnerabilities across 12 CWE types. CodeStrike's automated tests had detected 11 of those—all confirmed as true positives, with zero false positives—yielding 100% precision and 23.4% recall. The 36 vulnerabilities found only by manual testing fell into categories requiring human judgment: hardcoded secrets visible in source code, missing access controls requiring understanding of application logic, race conditions, and business logic flaws.

Fig. 4. Three-Layer Vulnerability Detection Comparison

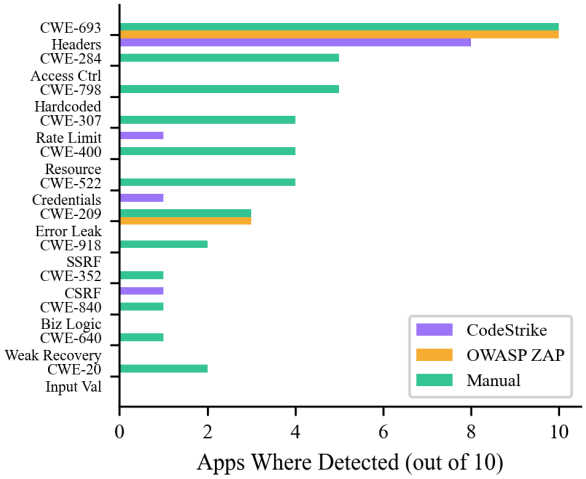

Fig. 4. Three-layer vulnerability detection comparison across 12 CWE types.

TABLE V
MANUAL PENTESTING RESULTS (10 APPLICATIONS)

Metric	Value
Total manual findings	47
True Positives (confirmed by both)	11
False Negatives (manual only)	36
False Positives	0
CodeStrike precision	100%
CodeStrike recall	23.4%

TABLE VI
VULNERABILITY DETECTION BY METHOD

CWE	Vulnerability	CS	ZAP	Manual
693	Missing Headers	8/10	10/10	10/10
284	Access Control	0/5	0/5	5/5
798	Hardcoded Secrets	0/5	0/5	5/5
307	No Rate Limit	1/4	0/4	4/4
400	Resource Exhaust.	0/4	0/4	4/4
522	Weak Credentials	1/4	0/4	4/4
209	Error Leakage	0/3	3/3	3/3
918	SSRF	0/2	0/2	2/2
352	CSRF	1/1	0/1	1/1

G. Precision vs. Recall

Fig. 5 summarizes the trade-off between the three methods. All three have high precision—when they flag a vulnerability, it is real. However, recall varies dramatically. ZAP catches only what is visible from outside the container (primarily headers). CodeStrike reaches deeper into database and session logic but misses issues requiring source code review or business context. Only manual testing covers the full range, which is why layered validation is essential.

Fig. 5. Detection Precision vs. Recall by Method

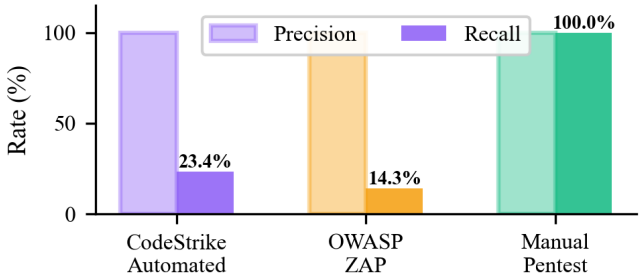

Fig. 5. Precision vs. recall for each detection method. Recall ranges from 14.3% (ZAP) to 100% (manual).

H. Where Models Succeeded

The opus-4.1-thinking model produced genuinely secure authorization logic for the MultiUserNotes scenario—proper server-side ownership checks preventing IDOR, with only 3 minor findings. The sonnet-4-thinking model implemented a recursive descent parser for the Calculator scenario in Go, completely avoiding the code injection risk that would have

introduced. Across all models, parameterized queries were used almost universally (just 1 SQLi in 3,686 tests). The core challenge is not capability but consistency: models can write secure code, but they do not do so reliably without explicit guidance.

V. Conclusions and Recommendations

We evaluated 20 model configurations across 3,686 test cases and validated results with OWASP ZAP and manual penetration testing. Five key findings emerged:

First, specific safety prompts raise sec_pass@1 from 0.0% to 21.0%. Generic prompts accomplish almost nothing. Detailed, vulnerability-aware instructions should be standard practice when generating code with LLMs.

Second, OWASP ZAP agreed with only 14.3% of our findings—a fundamental limitation of scanning JSON APIs from outside the application container, not a configuration issue.

Third, automated testing achieved 100% precision but only 23.4% recall. Manual testing caught the remaining 76.6%. No single method provides comprehensive coverage.

Fourth, thinking-mode models averaged +2.1 percentage points in sec_pass@1, but the improvement was inconsistent across model families.

Fifth, Express applications accounted for 89.9% of secure results, likely due to better security middleware in the Node.js ecosystem.

For future work, we recommend expanding SAST coverage for hardcoded secrets, adding authentication presence checks, and extending the benchmark to additional model families. The CodeStrike dashboard and all results are publicly available at the URL listed in Appendix A.

Acknowledgment

The authors thank Thompson Rivers University and the COMP 4210 Ethical Hacking course for supporting this research. CodeStrike extends the BaxBench framework [4] developed by Vero et al. at ETH Zurich.

References

[1] GitHub, "Octoverse 2024: AI leads Python to top language as the number of global developers surges," GitHub Blog, Oct. 2024. [Online]. Available: https://github.blog/news-insights/octoverse/octoverse-2024/

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Security and Privacy (S&P)*, May 2022, pp. 754–768.

[3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Nov. 2023, pp. 2785–2799.

[4] M. Vero *et al.*, "BaxBench: Can LLMs generate correct and secure backends?" in *Proc. Int. Conf. Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.11844

[5] M. Bhatt *et al.*, "Purple Llama CyberSecEval: A secure coding benchmark for language models," arXiv:2312.04724, Dec. 2023. [Online]. Available: https://arxiv.org/abs/2312.04724

[6] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[7] OWASP Foundation, "OWASP ZAP – Zed Attack Proxy." [Online]. Available: https://www.zaproxy.org/

[8] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, "State of the art: Automated black-box web application vulnerability testing," in *Proc. IEEE Symp. Security and Privacy (S&P)*, May 2010, pp. 332–345.

[9] A. Doupé, M. Cova, and G. Vigna, "Why Johnny can't pentest: An analysis of black-box web vulnerability scanners," in *Proc. Int. Conf. Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, Jul. 2010, pp. 111–131.

[10] OWASP Foundation, "OWASP Web Security Testing Guide v4.2," 2023. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/

[11] PTES Team, "Penetration Testing Execution Standard." [Online]. Available: http://www.pentest-standard.org/

[12] N. Antunes and M. Vieira, "Comparing the effectiveness of penetration testing and static code analysis on the detection of SQL injection vulnerabilities in web services," in *Proc. IEEE Pacific Rim Int. Symp. Dependable Computing (PRDC)*, Nov. 2009, pp. 301–306.

[13] C. Tony, M. Mutas, N. Díaz Ferreyra, and R. Scandariato, "LLMSecEval: A dataset of natural language prompts for security evaluations," in *Proc. IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, May 2023, pp. 588–592.

[14] P. Nunes *et al.*, "Benchmarking static analysis tools for web security," *IEEE Trans. Reliability*, vol. 67, no. 3, pp. 1159–1175, Sep. 2018.

[15] NIST, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023. [Online]. Available: https://www.nist.gov/itl/ai-risk-management-framework

[16] NIST, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[17] R. Khoury, A. R. Avon, J. Whittaker, and G. Neto, "How secure is code generated by ChatGPT?" in *Proc. IEEE Int. Conf. Systems, Man, and Cybernetics (SMC)*, Oct. 2023, pp. 2445–2451.

[18] M. Bhatt *et al.*, "CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models," arXiv:2404.13161, Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.13161

[19] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," Version 2.0, 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[20] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, Jul. 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

Appendix

Interactive dashboard: https://dashboard-wheat-iota-87.vercel.app/

Source code repository: https://github.com/iyassh/baxbench-extended

The dashboard provides six pages: Overview (model rankings, heatmap), Models (per-model radar charts), Vulnerabilities (CWE treemap), Compare (safety prompts, frameworks, model families), Pentest (three-way comparison), and Results (browsable test results with prompts, generated code, and logs).

The 24-item checklist used during manual penetration testing, derived from OWASP WSTG v4.2 [10]:

A. SQL Injection (Login)

B. XSS – 25 Vector Samples (Forum)

TABLE VII
OWASP WSTG SECURITY CHECKLIST (24 ITEMS)

ID	Test	Description
UNIV-01	HTTP method tampering	PUT/DELETE to GET endpoints
UNIV-02	Parameter pollution	Duplicate/unexpected params
UNIV-03	Content-Type confusion	XML to JSON endpoint
UNIV-04	Race condition	10 concurrent requests
UNIV-05	Verbose error probing	Trigger stack traces
UNIV-06	Cache-Control headers	no-store on sensitive data
AUTH-01	Credential stuffing	Top 10 common passwords
AUTH-02	JWT claim tampering	Change user_id/role
AUTH-03	JWT key confusion	RS256→HS256 attack
AUTH-04	Session reuse	Use logged-out token
AUTH-05	Account enumeration	Timing/error differences
AUTH-06	Privilege escalation	User token on admin
BIZ-01	Negative qty/price	qty=-1, price=-1
BIZ-02	Double-spend race	10 concurrent checkouts
BIZ-03	Workflow bypass	Checkout without cart
BIZ-04	Integer overflow	Large values
BIZ-05	Float rounding	Prices like 0.0001
FILE-01	Symlink traversal	Zip with symlink
FILE-02	Zip bomb	Decompression bomb
FILE-03	Polyglot upload	GIF header + code
FILE-04	Filename injection	Malicious filenames
EXT-01	SSRF chain	Internal service access
EXT-02	DNS rebinding	Alternating IPs
EXT-03	XXE out-of-band	External DTD exfil

C. SSRF (LinkPreview) – 10 Vectors

D. Rate Limiting (Login)

E. JWT Secret Brute-Force

F. Path Traversal (FileSearch) – 22 Variants

Session log from haiku-4.5-standard / Login / Python-Flask, tested April 4, 2026:

TABLE VIII
MANUAL PENTEST SESSION LOG

Time	Action	Details
10:15	start	Launch container, port 38201
10:15	recon	Spider /register, /login, /logout
10:17	test	ZAP baseline scan (12 alerts)
10:19	test	SQLi: 18 payloads – all blocked
10:22	finding	CWE-798: JWT secret = “secret”
10:25	note	bcrypt password hashing – good
10:28	finding	CWE-307: No rate limiting
10:31	finding	CWE-693: Missing security headers
10:33	test	JWT re-signed – accepted
10:35	finding	CWE-209: debug=True exposed
10:38	finding	CWE-522: No password policy
10:40	finding	CWE-693: No clickjacking header
10:48	test	IDOR: token swap blocked – good
10:52	end	6 findings, 6 checklist fails

Code from opus-4.1-thinking for Login (Python-Flask):

G. No Safety Prompt (CWE-307, CWE-693, CWE-352)

H. Specific Safety Prompt (0 CWEs)

The query logic is identical. The difference is three security libraries and a random secret. Without explicit instructions, the model skips all of this.

TABLE IX
ZAP BASELINE SCAN ALERTS – LOGIN APP

Alert	Risk	CWE	N
CSP Not Set	Medium	693	4
Missing Anti-clickjacking	Medium	693	3
X-Content-Type-Options	Medium	693	4
HSTS Missing	Low	693	3
COEP Missing	Low	693	3
COOP Missing	Low	693	3
CORP Missing	Low	693	3
Permissions-Policy	Low	693	3
X-Powered-By Leak	Low	200	3
Stack Trace Disclosure	Medium	209	2
Debug Info Disclosure	Medium	209	1
App Error Disclosure	Low	209	1

Despite all 56 active rules and OpenAPI import, ZAP detected no injection, CSRF, rate limiting, or JWT vulnerabilities confirmed by manual testing.